

ML Loop Code-Along - Step 06

Repeat with a built-in dataset

Incremental prerequisite prep for Imperial ML/AI coursework

Prepared for Anthony Watts - June 2026

Rule: nothing appears before it is explained.

6. Repeat with a built-in dataset

The tiny house table is good for seeing the workflow, but too small to trust. Now repeat the same loop using a built-in scikit-learn regression dataset. This avoids downloads and keeps the chapter runnable offline once packages are installed.

Step 6.1: Import a built-in regression dataset

Why this cell exists

`load_diabetes` gives us a small numeric dataset bundled with scikit-learn. Use it as a modelling toy, not as medical advice.

```
from sklearn.datasets import load_diabetes
```

What to look for

This import only gives access to a dataset-loading function.

Step 6.2: Load the dataset as pandas objects

Why this cell exists

`as_frame=True` asks scikit-learn to return pandas-friendly data, so you can inspect columns rather than stare at anonymous arrays.

```
diabetes = load_diabetes(as_frame=True)

X_reg = diabetes.data
y_reg = diabetes.target

print("X_reg:", X_reg.shape)
print("y_reg:", y_reg.shape)
X_reg.head()
```

What to look for

This is the same X/y pattern as before, just with more rows.

Step 6.3: Split the larger regression data

Why this cell exists

The same splitting helper works for the larger dataset. The model must still learn only from training data.

```
X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(
    X_reg,
    y_reg,
    test_size=0.25,
    random_state=42,
)
```

What to look for

The naming is verbose on purpose. Long names are useful while learning because they reduce accidental train/test mix-ups.

Step 6.4: Fit and predict again

Why this cell exists

We reuse LinearRegression because we are still predicting a number. The workflow is the same: create, fit, predict.

```
reg_model = LinearRegression()
reg_model.fit(X_reg_train, y_reg_train)

reg_train_predictions = reg_model.predict(X_reg_train)
reg_test_predictions = reg_model.predict(X_reg_test)
```

What to look for

Notice that there are separate predictions for training rows and test rows. That lets us compare fit vs generalisation.

Step 6.5: Compare train and test error

Why this cell exists

If training error is much lower than test error, the model may be overfitting. Here we start with MAE.

```
train_mae = mean_absolute_error(
    y_reg_train,
    reg_train_predictions,
)

test_mae = mean_absolute_error(
    y_reg_test,
    reg_test_predictions,
)

print("Train MAE:", round(train_mae, 2))
print("Test MAE:", round(test_mae, 2))
```

What to look for

The exact values are less important than the comparison. Training performance alone is not a trustworthy estimate of usefulness.

Step 6.6: Add R-squared carefully

Why this cell exists

R-squared gives another view of regression performance. It is common, but not magical. Higher is usually better, but always interpret it in context.

```
from sklearn.metrics import r2_score
```

What to look for

We import this only now because this is the first point where we need it.

Step 6.7: Calculate train and test R-squared

Why this cell exists

This makes the train-vs-test comparison visible using a second metric.

```
train_r2 = r2_score(y_reg_train, reg_train_predictions)
test_r2 = r2_score(y_reg_test, reg_test_predictions)

print("Train R2:", round(train_r2, 3))
print("Test R2:", round(test_r2, 3))
```

What to look for

A model can score well on training data and poorly on test data. That gap is the practical smell of overfitting.